Flask Web Dashboard

This document describes the web dashboard's architecture, the Flask concepts applied, and how the backend communicates with the frontend.

Overview

The web dashboard is a server-side rendered Flask application. It provides administrators with analytics, sign-in logs, people management, and data export functionality through a standard web browser.

The dashboard code lives in the `dashboard/` directory: - `app.py` -- All routes, logic, and database queries. - `templates/` -- Jinja2 HTML templates. - `static/style.css` -- Stylesheet (mobile first; see Responsive Layout). - `static/layout.js` -- Shared page behaviour loaded by `base.html`: the burger menu and stacked-table labels.

Flask Application Setup

The Flask app is created with standard configuration:

```
app = Flask(__name__)
app.secret_key = _resolve_secret_key()   # from DASHBOARD_SECRET_KEY env var, or a random fallback
```

The secret key is used for signing session cookies -- see 09-authentication-and-security.md for how it's sourced, plus CSRF protection (`flask-wtf`), login rate limiting (`flask-limiter`), and secure cookie settings, all configured at app-creation time alongside it. When running from a PyInstaller bundle, the template and static directories are resolved relative to the bundle's temporary extraction directory (`sys._MEIPASS`).

`init_db()` is called once at module import time (not just from the kiosk's `app.py`), so the dashboard can also be started completely standalone -- without the kiosk ever having run first -- and still have a working schema and a seeded login.

Routing

Flask uses decorators to map URL paths to Python functions. The dashboard defines these routes:

Route	Method	Access	Purpose
`/`	GET	any logged in	Redirect to dashboard (admin) or sign-in (staff)

`/login`	GET, POST	public	Login form and authentication (rate-limited)
`/logout`	GET	public	Clear session and redirect
`/signin`	GET, POST	any logged in	Web Sign In / Out page and submission
`/signin/search`	GET	any logged in	JSON: search members/staff by name for the sign-in page
`/signin/alerts`	GET	any logged in	JSON: active alerts for a set of member ids
`/signin/currently-in`	GET	any logged in	JSON: everyone currently signed in, for the sign-in page's bulk-select action
`/status`	GET	any logged in	Read-only "who's here" view (staff's home page; admins can also reach it)
`/my-timesheet`	GET	any logged in	Self-service view of the logged-in staff member's own sign-in history
`/my-timesheet/download`	GET	any logged in	Download the logged-in staff member's own timesheet (`fmt=xlsx|pdf|csv`, `date_from`/`date_to` optional)
`/run-sheets`	GET	any logged in	List/search run sheets, with each one's member count
`/run-sheets/new`	GET, POST	any logged in	Create a new run sheet (name + member picker)
`/run-sheets/<int:run_id>/edit`	GET, POST	any logged in	Rename a run sheet and edit its membership
`/run-sheets/<int:run_id>/delete`	POST	any logged in	Delete a run sheet

`/run-sheets/search`	GET	any logged in	JSON: search run sheets by name (used by the sign-in page's "Load a Run Sheet")
`/run-sheets/<int:run_id>/members`	GET	any logged in	JSON: a run sheet's members, as `{id, name}` pairs
`/dashboard`	GET	admin	Analytics overview
`/logs`	GET	admin	Paginated sign-in log, with status/search/date filters
`/logs/sign/<int:sign_id>/edit`	GET, POST	admin	Edit an existing sign-in/out time
`/logs/sign/add`	GET, POST	admin	Add a missing sign-in/out time (`member_id`, `direction`, `pair_id` = the existing other half of the visit)
`/logs/sign/delete`	POST	admin	Delete one or more sign-in/out records (`sign_id`, repeatable): a whole visit from the logs table, or one record from the edit page
`/audit`	GET	admin	Paginated audit log
`/people`	GET	admin	Member or staff listing
`/people/new`	GET, POST	admin	Create a new person
`/people/<int:person_id>`	GET, POST	admin	Edit an existing person
`/people/<int:person_id>/delete`	POST	admin	Delete a person
`/alerts`	GET, POST	admin	View/create sign-in alerts
`/alerts/<int:alert_id>/delete`	POST	admin	Delete an alert
`/exports`	GET	admin	Export page

`/export/staff`	GET	admin	Download staff file -- all staff (`date_from`/`date_to` required) or one staff member (`person_id`, dates optional); `fmt=xlsx|pdf|csv`, defaults to xlsx
`/export/person`	GET	admin	Download one person's file (`person_id` required; `date_from`/`date_to` optional; `fmt=xlsx|pdf|csv`, defaults to xlsx)
`/people/<int:person_id>/signature.png`	GET	admin	Stream that person's most recently used signature as a PNG (404 if none on file)
`/dashboard-users`	GET	admin	List dashboard login accounts
`/dashboard-users/new`	POST	admin	Create a dashboard login (name + password + role + optional linked staff member)
`/dashboard-users/<int:user_id>/edit`	GET, POST	admin	Rename, re-role, re-link, or change the password of a dashboard login
`/dashboard-users/<int:user_id>/delete`	POST	admin	Delete a dashboard login (blocked for the last remaining admin)

Authentication

The dashboard authenticates against the `dashboard_users` table (name + hashed password), not a hardcoded credential -- see 09-authentication-and-security.md for the full design (hashing, password rules, roles, account lockout, CSRF, rate limiting, security headers, secure cookies). In short:

1. User submits name and password via the login form (rate-limited to 5 attempts/minute per IP, plus a per-account lockout after 5 consecutive failures).
2. The server checks `database.is_dashboard_user_locked(name)` first, then looks up `database.check_dashboard_login(name, pin)`.

3. On success, the session stores `user_id`, `name`, `role`, and `member_id` (not just a boolean) -- `member_id` is `None` unless an admin has linked this login to a staff record.
4. Two decorators guard routes: `login_required` (any authenticated user) and `admin_required` (must additionally have `role == "admin"`).

```
def login_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        if not session.get("user_id"):
            return redirect(url_for("login"))
        return f(*args, **kwargs)
    return decorated


def admin_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        if not session.get("user_id"):
            return redirect(url_for("login"))
        if session.get("role") != "admin":
            flash("Admin access required.", "error")
            return redirect(url_for("signin"))
        return f(*args, **kwargs)
    return decorated
```

The `@wraps(f)` decorator from `functools` preserves the original function's name and docstring, which Flask needs for `url_for()` to work correctly.

Template Rendering (Jinja2)

Flask uses Jinja2 as its template engine. Templates live in `dashboard/templates/` and use template inheritance:

Base Template (`base.html`)

Defines the page skeleton: HTML head, navigation bar, flash message area, and a content block. All other templates extend this:

```
{% extends "base.html" %}
{% block content %}
 <!-- page-specific content -->
{% endblock %}
```

Template Variables

Python data is passed to templates via `render_template()`:

```
return render_template(
    "dashboard.html",
```

```
    today_ins=today_ins,
    daily_labels=json.dumps(daily_labels),
    daily_values=json.dumps(daily_values),
    # ...
)
```

Lists and dictionaries intended for JavaScript (chart data) are serialised to JSON with `json.dumps()` in Python and then parsed in the template with Jinja2's `| safe` filter or `| tojson`.

Flash Messages

The dashboard uses Flask's `flash()` system for one-time status messages:

```
flash("Changes saved.", "success")
flash("First name and last name are required.", "error")
```

These are rendered in the base template after redirect, providing user feedback for form submissions.

Database Access

The dashboard has its own `get_conn()` function that connects to the same SQLite database as the kiosk. For operations that involve the kiosk's data models (creating/updating members, caregivers), it imports functions directly from the kiosk's `database.py` module:

```
from database import upsert_member, upsert_caregiver
```

This avoids duplicating business logic. The imports are done inside route functions (lazy imports) rather than at module level to avoid import order issues when both the kiosk and dashboard initialise.

Analytics Queries

The dashboard route performs several aggregate queries to compute statistics:

Currently Signed In

This is computed by `database.get_currently_signed_in()` (in `kiosk_signin/database.py`, not `app.py`), a single shared function rather than dashboard-only SQL, since both the dashboard and the sign-in page's "Select all currently signed in" bulk action need the same "who's here right now" answer. It uses a correlated subquery to find people whose most recent sign event today has direction "IN":

```
SELECT m.id, m.first_name, m.last_name, s.timestamp
FROM members m
JOIN sign_ins s ON s.member_id = m.id
WHERE s.timestamp LIKE ?
AND s.direction = 'IN'
```

```
AND s.id = (
   SELECT s2.id FROM sign_ins s2
   WHERE s2.member_id = m.id AND s2.timestamp LIKE ?
   ORDER BY s2.timestamp DESC, s2.id DESC LIMIT 1
)
```

This finds each person's latest sign event today (by timestamp, not insertion order, so a back-dated record added from the logs page doesn't count as their newest) and returns only those where the direction is IN, including the `id` needed to pre-select them on the sign-in page. `dashboard/app.py`'s `_today_snapshot()` helper calls this function and derives its "currently in" count from `len(...)`, rather than running a second, separate COUNT query.

Daily Totals

Groups sign events by date for the last 30 days. The Python code then fills in gaps (days with zero events) to produce a continuous series for the chart:

```
for i in range(30, -1, -1):
   d = (date.today() - timedelta(days=i)).isoformat()
   daily_labels.append(d[5:])
   daily_values.append(daily_map.get(d, 0))
```

Peak Hours

Uses SQLite's `strftime('%H', timestamp)` to extract the hour from each sign event and groups by hour across all time.

Weekly and Monthly Totals

Uses `strftime('%Y-W%W', timestamp)` and `strftime('%Y-%m', timestamp)` respectively to group sign events into week and month buckets.

Pagination

The logs, audit, and people pages implement server-side pagination:

```
page = max(1, int(request.args.get("page", 1)))
per_page = 50
offset = (page - 1) * per_page
total_pages = max(1, (total + per_page - 1) // per_page)
```

The total count is queried first, then the page's rows are fetched with `LIMIT ? OFFSET ?`. Page navigation links are generated in the template.

Filtering

The logs page supports filtering by name search, date range and status using query parameters. Only the name search is applied at the SQL level; the date range and status filters are applied afterwards in Python to the visits built by `_pair_sign_in_visits`, since both are properties of a paired *visit*, not a single `sign_ins` row (filtering rows by date first would orphan the OUT of a visit whose IN falls outside the range). A visit is dated by its IN time, or its OUT time if it has no IN. The SQL-level condition is dynamically built:

```
where_parts = []
params = []
if search:
   where_parts.append("(m.first_name || ' ' || m.last_name) LIKE ?")
   params.append(f"%{search}%")
where_clause = ("WHERE " + " AND ".join(where_parts)) if where_parts else ""
```

This approach safely constructs SQL WHERE clauses while still using parameterised queries to prevent injection. After the matching rows are paired into visits, `date_from`/`date_to` and then `status=OPEN`/`status=CLOSED` filter the resulting list.

Editing, Adding and Deleting Records

Visits aren't stored — `_pair_sign_in_visits` rebuilds them on every page load by walking each person's records in time order and pairing each IN with the OUT that follows it. That means a record's *time* decides which visit it belongs to, so the edit and add pages only accept a time that sits strictly between the neighbouring records (`database.get_sign_neighbours()`):

- **Edit** — the new time must be after the person's previous record and before their next one.
- **Add** — the link from the logs table passes `pair_id`, the existing half of the visit. A missing OUT must fall after its IN and before the person's next record; a missing IN must fall before its OUT and after the previous record.

Without this, moving or adding an OUT earlier than its IN reorders the records and the logs page shows one visit as two separate rows. The allowed range is shown under the time field (and set as the input's `min`/`max`); anything outside it is rejected with a flash message.

`/logs/sign/delete` removes records via `database.delete_signs()` and writes a `SIGN_DELETED` audit entry per record. The logs table has a **Delete** button per visit (both halves), and the edit page has one for just that record. Works the same for members and staff.

File Downloads

The export routes generate files in memory and send them to the browser with `send_file()`. `_export_person_response(person, date_from, date_to, fmt="xlsx")` is the shared helper behind `/export/person`, `/export/staff`'s single-person branch, and `/my-timesheet/download` -- it dispatches on `fmt` to pick the right generator and MIME type from `export_util.py`:

```
_EXPORT_FORMATS = {
   "xlsx": "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
   "pdf": "application/pdf",
   "csv": "text/csv",
}

if fmt == "pdf":
   buf = io.BytesIO()
   (generate_staff_export_pdf if person_type == "staff" else generate_export_pdf)(
       signs, export_from, export_to, buf)
elif fmt == "csv":
   text_buf = io.StringIO()
   (generate_staff_export_csv if person_type == "staff" else generate_export_csv)(
       signs, export_from, export_to, text_buf)
   buf = io.BytesIO(text_buf.getvalue().encode("utf-8"))
else:
   buf = io.BytesIO()
   (generate_staff_export if person_type == "staff" else generate_export)(
       signs, export_from, export_to, buf)
buf.seek(0)
return send_file(buf, as_attachment=True, download_name=filename, mimetype=_EXPORT_FORMATS[fmt])
```

An unrecognised `fmt` value falls back to `xlsx` rather than erroring. CSV is generated with the stdlib `csv` module into a text buffer, then encoded to bytes for `send_file` (which expects a binary stream). PDF is generated with `xhtml2pdf` (already a project dependency, previously only used by `Docs/generate_printable_docs.py` -- see Run Sheets (Web) for another example of reusing an existing tool rather than adding a new one), rendering a small HTML table per person and writing straight to a `BytesIO`. The `as_attachment=True` flag tells the browser to download the file rather than displaying it, and the filename's extension is derived from `fmt` (`KioskSignIn_{name}_Export_{from}_{to}.{fmt}`).

`/export/staff`'s bulk (no `person_id`) branch isn't routed through `_export_person_response` -- it queries `get_staff_signs_for_export()` directly for every staff member at once -- so it repeats the same three-way `fmt` dispatch inline rather than through the shared helper.

Responsive Layout

The stylesheet is mobile first: the base rules target phones and `min-width` media queries add to them at 600px, 720px and 900px.

- **Navigation** — collapsed behind a burger button by default. `static/layout.js` measures whether every tab fits on one line and, only if it does, adds `.nav-wide` to the header to show the inline tab bar. Admins (who have the most tabs) get the burger on small laptops; staff, with only a few tabs, get the inline bar sooner. Esc closes the menu. Logout sits at the end of the menu.
- **Tables** — any `<table>` inside `.table-wrap` renders as a stack of cards below 720px, one card per row with each cell labelled. `layout.js` copies each column's `<th>` text into its cells' `data-label` attribute, so new tables get this for free; cells under a blank header (action buttons) are right-aligned without a label, and a `cell-title` class on a cell shows it as the card heading with no label.
- **Inputs** use a 16px font so iOS doesn't zoom the page on focus.
- **Charts** sit in fixed-height `.chart-box` containers with `maintainAspectRatio: false`, so they keep a readable height on narrow screens.

Charts (Client-Side)

The dashboard page uses Chart.js (loaded from a CDN in the template) to render interactive charts. Chart data is injected into the HTML as JSON:

```
<script>
 const dailyLabels = {{ daily_labels | safe }};
 const dailyValues = {{ daily_values | safe }};
 new Chart(ctx, { type: 'bar', data: { labels: dailyLabels, datasets: [...] } });
</script>
```

The `| safe` filter tells Jinja2 not to escape the JSON string, since it was already serialised safely in Python.

Form Handling

People creation and editing use standard HTML forms with POST method. Flask's `request.form` dictionary provides access to submitted values:

```
data = {
   "first_name": request.form.get("first_name", "").strip(),
   "last_name":  request.form.get("last_name", "").strip(),
   # ...
}
```

Multi-value form fields (like day checkboxes) use `request.form.getlist()` and are joined into comma-separated strings for storage.

After a successful save, the pattern is Post/Redirect/Get (PRG): the route processes the form, then redirects to the edit page. This prevents duplicate submissions if the user refreshes the page.

Members vs. Staff (`person_type`)

Members and staff are both rows in the same `members` table, distinguished by a `person_type` column (`"member"` or `"staff"`; `NULL` is treated as `"member"` for rows created before the column existed). A single template, `person_form.html`, renders both the "new"/"edit member" screen and the "new"/"edit staff" screen -- it switches between the full member form (child details, caregiver, attendance days, activities) and the short staff form (name, role, PIN) based on a `person_type` variable passed in from the route.

Because both screens share one `<form>`, `person_type` has to be threaded through explicitly as a hidden field:

```
<input type="hidden" name="person_type" value="{{ person_type }}">
```

On the **create** path (`/people/new`), this hidden field is the *only* source of truth for which type is being created -- `app.py`'s `person_new()` reads it with `request.form.get("person_type", "member")`. If that hidden input is ever removed or renamed, every new "staff" submission silently defaults to `"member"`

regardless of which tab (Members or Staff) the admin was on, and the post-create redirect then renders the member form instead of the staff form for the record just created. This exact bug shipped once; see `10-testing.md` for the regression tests that now guard against it.

On the **edit** path (`/people/<id>`), `person_type` is *not* read from the form at all -- `person_edit()` looks it up from the existing database row (`person.get("person_type")`) and reuses that value, since a person's type isn't meant to change after creation. The hidden field is rendered on the edit screen too (for template consistency) but has no effect on the POST handler.

Web Sign In / Out Page

`/signin` reimplements the kiosk's sign-in/out flow (`kiosk_signin/screens/signature_screen.py`) as a web page, reusing the exact same data-layer functions rather than duplicating the logic: `get_allowed_direction()`, `record_sign()`, `get_alerts_for_members()`, and `email_notifier.send_notification()`. The route, `signin.html`, and `static/signin.js` work together:

1. `signin.js` debounces keystrokes in the search box and calls `GET /signin/search?type=member|staff&q=...`, which returns JSON `{"results": [{"id": ..., "name": ...}]}` from `search_members()`/`search_staff()`.
2. Selecting a result adds them to an in-page "selected" list (chips), and triggers `GET /signin/alerts?member_ids=1,2,3` -- if any alerts come back, a modal shows them before the direction/notes/signature section is revealed, mirroring the kiosk's pre-submit alert popup.
3. On submit, the browser POSTs to `/signin` with repeated `member_ids` fields, `direction`, `notes`, and `signature_data`.

The search-and-select behaviour (steps 1-2's chip list) is factored out into `dashboard/static/person-picker.js`, a small shared widget instantiated with `window.PersonPicker.init({...})`. `signin.js` and `run_sheet_form.js` (see Run Sheets (Web) below) each create their own picker instance pointed at their own DOM elements, so the same search/select/chip code isn't duplicated between the two pages.

Two bulk-select shortcuts sit above the search box on the sign-in page, both of which add people to the same picker instance rather than replacing the selection, so ad hoc individuals can still be added on top of either one:

- **Load a Run Sheet** -- a second, smaller search box that queries `GET /run-sheets/search?q=...` (run sheet names, not people). Clicking a result fetches `GET /run-sheets/<id>/members` and adds every member of that run sheet as a chip.
- **Select all currently signed in** -- a button that fetches `GET /signin/currently-in` (backed by `get_currently_signed_in()`, see Currently Signed In above), adds everyone returned as a chip, and then programmatically clicks the existing "OUT" direction button, since signing someone out is the only valid next action for a person who is already signed in. Staff can still switch back to IN by hand if that was a mistake.

Signature Format Bridging

The kiosk's `SignatureCanvas` stores a signature as a Python list of `(x1, y1, x2, y2)` line-segment tuples, serialised with `str(strokes).encode()` -- and `export_util.render_signature()` decodes it back with `ast.literal_eval()`. For the web signature pad to produce signatures that exports and the `/people/<id>/signature.png` route can read without any changes, `/signin`'s POST handler converts the browser's stroke data into that exact byte format server-side:

```
def _parse_signature(raw: str):
    strokes = json.loads(raw)                       # browser sends JSON: [[x1,y1,x2,y2], ...]
    ...                                              # validated: must be a list of 4-number lists
    return str([tuple(s) for s in clean]).encode()  # same format as SignatureCanvas.export_bytes()
```

The browser never builds the Python-literal string itself -- it only ever sends plain JSON, which is what browsers can serialise losslessly. Converting and validating server-side means malformed or oversized input (capped at 20,000 segments) is rejected with a flash message rather than silently producing a blob that would later fail `ast.literal_eval()` at export time. A signature is optional, matching the kiosk (which no longer requires one either) -- an empty `signature_data` field just results in `signature=None` being passed to `record_sign()`.

Direction Validation and Partial Success

Just like the kiosk, submitting a batch of people can partially succeed: each selected person is checked individually against `get_allowed_direction()`, so if one person in a group is already signed in and the rest aren't, only the eligible ones are recorded and the flash message reports how many were skipped -- nobody's submission is rejected outright just because one person in the group wasn't eligible.

Run Sheets (Web)

The kiosk has managed run sheets (named, reusable groups of people, e.g. a school bus run) since before the web dashboard existed, using `search_runs()`, `get_run_members()`, and `upsert_run()` in `kiosk_signin/database.py`. Those functions are name-keyed: `upsert_run(name, member_ids)` looks up a run by its `name`, so it can create or replace membership but cannot safely rename an existing run without either colliding with the row it meant to update or leaving a duplicate `run_groups` row behind.

Full run sheet management on the dashboard (`/run-sheets` and friends) needed proper rename support, so it uses a separate, id-keyed pair of functions instead:

```
def create_run(name: str, member_ids: list):
    """Insert-only. Returns None instead of raising if the name is
    already taken (run_groups.name is UNIQUE)."""
    try:
        with get_conn() as conn:
            cur = conn.execute("INSERT INTO run_groups (name) VALUES (?)", (name,))
            run_id = cur.lastrowid
            ...
        return run_id
    except sqlite3.IntegrityError:
        return None
```

```
def update_run(run_id: int, name: str, member_ids: list) -> bool:
    """Renames by id. Returns False instead of raising on a name
    collision with a *different* run, or if run_id doesn't exist."""
    try:
        with get_conn() as conn:
            cur = conn.execute("UPDATE run_groups SET name=? WHERE id=?", (name, run_id))
            if cur.rowcount == 0:
                return False
            ...
        return True
    except sqlite3.IntegrityError:
        return False
```

The `/run-sheets/new` and `/run-sheets/<id>/edit` routes check for `None`/`False` and flash `A run sheet named "..." already exists.` instead of letting the `sqlite3.IntegrityError` reach the user as a 500 error. The kiosk's own `upsert_run()`/`delete_run()` are untouched behaviourally, aside from gaining `log_event()` calls (`RUN_CREATED`/`RUN_UPDATED`/`RUN_DELETED`) so run sheet changes made from either the kiosk or the dashboard show up in the audit log, matching the pattern already used by `upsert_member()`/`delete_member()`.

`run_sheets.html` (the list page) and `run_sheet_form.html` (create/edit) follow the same templates used for people management -- `run_sheets.html` mirrors `people.html`'s search-plus-table layout, and `run_sheet_form.html` embeds a `person-picker.js` instance (see Web Sign In / Out Page above) pre-populated with the run's current members on the edit page via a small inline `<script type="application/json">` block, so `run_sheet_form.js` can read the initial selection without an extra round trip.

My Timesheet (Self-Service)

`/my-timesheet` and `/my-timesheet/download` let a staff member view and download their own sign-in history without needing admin access. Both routes resolve the person strictly from the session, never from a request parameter:

```
user = get_dashboard_user(session["user_id"])
if not user or not user.get("member_id"):
    flash("Your account isn't linked to a staff record yet. Ask an admin to link it "
          "from the Manage Dashboard Users page.", "error")
    return redirect(url_for("signin"))
```

This is deliberate: `person_id`/`member_id` are never read from `request.args` on these two routes, unlike the admin export routes where an admin legitimately chooses whose records to export. A staff account tampering the URL to add `?person_id=...` has no effect -- there's no code path in `my_timesheet()`/`my_timesheet_download()` that would ever read it.

`my_timesheet()` reuses `get_signs_for_member()` and `_pair_sign_in_visits()` (the same pairing function the admin Logs page uses -- see Filtering above) to render an on-page table of the member's sessions, with an optional `date_from`/`date_to` query string filter matching the exact same optional-range UX as the admin "Export a Person's Sign-ins" page. `my_timesheet_download()` calls

`_export_person_response()` (see File Downloads above) with the same date range and a `fmt` choice, so a staff member gets the identical xlsx/pdf/csv output an admin would get exporting them.

`get_signs_for_member()`'s SELECT was widened to include `sign_ins.id` and `sign_ins.notes` (previously omitted) specifically so its output could feed `_pair_sign_in_visits()` -- both are backward-compatible additions to the returned dict, not a breaking change for its other two callers (the kiosk's own export screen, and `_export_person_response`).